

用户认证与权限方案

2026年03月14日 v1.0

用户认证与权限方案

文档版本：v1.0

编写日期：2026 年 3 月 14 日

开发负责人：王舟

一、需求分析

1.1 核心需求

需求	说明	优先级
用户注册	邮箱/手机号注册	P0
用户登录	账号密码登录、记住登录	P0
密码管理	修改密码、找回密码	P0
角色权限	管理员/主办律师/律师助理	P0
案件权限	案件级访问控制	P0
Token 管理	JWT、刷新 Token	P0
登录日志	记录登录时间、IP	P1
多端登录	支持多设备同时登录	P2

1.2 角色定义

角色	权限	说明
管理员	全部权限	用户管理、系统配置
主办律师	案件管理、团队管理	创建案件、分配团队成员
律师助理	案件编辑、文件上传	协助主办律师处理案件
访客	只读权限	仅限查看指定案件

1.3 权限模型

RBAC（基于角色的访问控制）

用户 → 角色 → 权限
↓
案件权限（数据级权限）

二、数据库设计

2.1 核心表结构

```
-- 用户表
CREATE TABLE users (
  id          TEXT PRIMARY KEY,
  username    TEXT UNIQUE NOT NULL,
  email       TEXT UNIQUE NOT NULL,
  phone       TEXT,
  password_hash TEXT NOT NULL,

  -- 个人信息
  real_name   TEXT,
  avatar_url  TEXT,
  organization TEXT,
  position    TEXT,
```

```

-- 状态
status          TEXT DEFAULT 'active', -- active/inactive/banned
email_verified  BOOLEAN DEFAULT FALSE,
phone_verified   BOOLEAN DEFAULT FALSE,
last_login_at   DATETIME,
last_login_ip    TEXT,

created_at       DATETIME DEFAULT CURRENT_TIMESTAMP,
updated_at       DATETIME DEFAULT CURRENT_TIMESTAMP,

INDEX idx_email ON users(email),
INDEX idx_username ON users(username),
INDEX idx_status ON users(status)
);

-- 角色表
CREATE TABLE roles (
    id            TEXT PRIMARY KEY,
    code          TEXT UNIQUE NOT NULL, -- admin/host_lawyer/lawyer_assistant/guest
    name          TEXT NOT NULL,
    description    TEXT,
    permissions    TEXT, -- JSON 数组, 如 ["case:create", "case:edit"]
    created_at     DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 用户角色关联表
CREATE TABLE user_roles (
    id            TEXT PRIMARY KEY,
    user_id       TEXT NOT NULL REFERENCES users(id),
    role_id       TEXT NOT NULL REFERENCES roles(id),
    granted_by    TEXT NOT NULL,
    granted_at    DATETIME DEFAULT CURRENT_TIMESTAMP,
    expires_at    DATETIME, -- 角色有效期 (可选)

    UNIQUE INDEX idx_unique (user_id, role_id),
    INDEX idx_user ON user_roles(user_id),
    INDEX idx_role ON user_roles(role_id)
);

-- 案件成员表 (案件级权限)
CREATE TABLE case_members (
    id            TEXT PRIMARY KEY,
    case_id       TEXT NOT NULL REFERENCES cases(id),
    user_id       TEXT NOT NULL REFERENCES users(id),
    role_in_case  TEXT NOT NULL, -- owner/member/viewer
    joined_at     DATETIME DEFAULT CURRENT_TIMESTAMP,

```

```

        joined_by          TEXT NOT NULL,

        UNIQUE INDEX idx_unique (case_id, user_id),
        INDEX idx_case ON case_members(case_id),
        INDEX idx_user ON case_members(user_id)
    );

-- 登录日志表
CREATE TABLE login_logs (
    id                    TEXT PRIMARY KEY,
    user_id              TEXT NOT NULL REFERENCES users(id),
    login_time           DATETIME DEFAULT CURRENT_TIMESTAMP,
    ip_address           TEXT,
    user_agent           TEXT,
    login_status         TEXT NOT NULL, -- success/failed
    failure_reason       TEXT,         -- 失败原因

    INDEX idx_user ON login_logs(user_id),
    INDEX idx_time ON login_logs(login_time)
);

-- 权限字典表
CREATE TABLE permissions (
    id                    TEXT PRIMARY KEY,
    code                 TEXT UNIQUE NOT NULL, -- case:create, case:edit, file:upload
    name                 TEXT NOT NULL,
    module               TEXT NOT NULL, -- case/file/node/person
    created_at           DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- 初始化角色和权限
INSERT INTO roles (id, code, name, description) VALUES
('role_admin', 'admin', '管理员', '系统管理员, 拥有全部权限'),
('role_host', 'host_lawyer', '主办律师', '案件负责人, 管理案件和团队'),
('role_assistant', 'lawyer_assistant', '律师助理', '协助处理案件'),
('role_guest', 'guest', '访客', '只读权限');

INSERT INTO permissions (id, code, name, module) VALUES
-- 案件权限
('perm_001', 'case:create', '创建案件', 'case'),
('perm_002', 'case:view', '查看案件', 'case'),
('perm_003', 'case:edit', '编辑案件', 'case'),
('perm_004', 'case:delete', '删除案件', 'case'),
('perm_005', 'case:manage_members', '管理案件成员', 'case'),
-- 文件权限
('perm_010', 'file:upload', '上传文件', 'file'),
('perm_011', 'file:view', '查看文件', 'file'),

```

```
('perm_012', 'file:download', '下载文件', 'file'),
('perm_013', 'file:delete', '删除文件', 'file'),
-- 节点权限
('perm_020', 'node:create', '创建节点', 'node'),
('perm_021', 'node:edit', '编辑节点', 'node'),
('perm_022', 'node:delete', '删除节点', 'node'),
-- 人物权限
('perm_030', 'person:create', '创建人物', 'person'),
('perm_031', 'person:edit', '编辑人物', 'person');
```

三、认证流程设计

3.1 JWT Token 结构

```
// src/models/auth.rs
use chrono::{DateTime, Utc, Duration};
use serde::{Deserialize, Serialize};

/// JWT Claims
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Claims {
    /// 用户 ID
    pub sub: String,
    /// 用户名
    pub username: String,
    /// 角色列表
    pub roles: Vec<String>,
    /// 过期时间
    pub exp: usize,
    /// 签发时间
    pub iat: usize,
    /// Token 类型
    pub token_type: TokenType,
}

/// Token 类型
#[derive(Debug, Clone, Serialize, Deserialize, PartialEq)]
pub enum TokenType {
    Access,      // 访问 Token (短期)
    Refresh,     // 刷新 Token (长期)
}
```

```

impl Claims {
    /// 创建访问 Token
    pub fn new_access(user_id: String, username: String, roles: Vec<String>) -> Self {
        let now = Utc::now();
        let exp = now + Duration::minutes(60); // 1 小时有效期

        Self {
            sub: user_id,
            username,
            roles,
            exp: exp.timestamp() as usize,
            iat: now.timestamp() as usize,
            token_type: TokenType::Access,
        }
    }

    /// 创建刷新 Token
    pub fn new_refresh(user_id: String) -> Self {
        let now = Utc::now();
        let exp = now + Duration::days(7); // 7 天有效期

        Self {
            sub: user_id,
            username: String::new(),
            roles: Vec::new(),
            exp: exp.timestamp() as usize,
            iat: now.timestamp() as usize,
            token_type: TokenType::Refresh,
        }
    }
}

```

3.2 认证服务

```

// src/services/auth_service.rs
use argon2::{password_hash::PasswordHash, Argon2, PasswordVerifier};
use jsonwebtoken::{encode, decode, Header, Algorithm, Validation};
use crate::models::auth::*;

pub struct AuthService {
    pool: SqlitePool,
    secret: String,
}

impl AuthService {

```

```

pub fn new(pool: SqlitePool, secret: String) -> Self {
    Self { pool, secret }
}

/// 用户登录
pub async fn login(
    &self,
    username: &str,
    password: &str,
    ip_address: Option<&str>,
    user_agent: Option<&str>,
) -> Result<LoginResponse, AuthError> {
    // 1. 查询用户
    let user = self.find_user_by_username(username).await?
        .ok_or(AuthError::InvalidCredentials)?;

    // 2. 验证密码
    let parsed_hash = PasswordHash::new(&user.password_hash)
        .map_err(|_| AuthError::InvalidCredentials)?;

    let valid = Argon2::default()
        .verify_password(password.as_bytes(), &parsed_hash)
        .is_ok();

    if !valid {
        // 记录登录失败
        self.log_login(&user.id, false, ip_address, user_agent, Some("密码错误")).await?;
        return Err(AuthError::InvalidCredentials);
    }

    // 3. 检查用户状态
    if user.status != "active" {
        return Err(AuthError::UserInactive);
    }

    // 4. 获取用户角色
    let roles = self.get_user_roles(&user.id).await?;

    // 5. 生成 Token
    let access_token = self.generate_access_token(&user.id, &user.username, &roles)?;
    let refresh_token = self.generate_refresh_token(&user.id)?;

    // 6. 更新最后登录时间
    self.update_last_login(&user.id, ip_address).await?;

    // 7. 记录登录日志
    self.log_login(&user.id, true, ip_address, user_agent, None).await?;
}

```

```

        Ok(LoginResponse {
            user: UserInfo::from_user(user),
            access_token,
            refresh_token,
            expires_in: 3600,
        })
    }

/// 刷新 Token
pub async fn refresh_token(&self, refresh_token: &str) -> Result<TokenResponse, AuthError> {
    // 验证刷新 Token
    let claims = self.verify_token(refresh_token)?;

    if claims.token_type != TokenType::Refresh {
        return Err(AuthError::InvalidToken);
    }

    // 生成新的访问 Token
    let user = self.find_user_by_id(&claims.sub).await?
        .ok_or(AuthError::UserNotFound)?;

    let roles = self.get_user_roles(&user.id).await?;
    let access_token = self.generate_access_token(&user.id, &user.username, &roles)?;

    Ok(TokenResponse {
        access_token,
        expires_in: 3600,
    })
}

/// 验证 Token
pub fn verify_token(&self, token: &str) -> Result<Claims, AuthError> {
    let validation = Validation::new(Algorithm::HS256);
    let decoded = decode::<Claims>(
        token,
        self.secret.as_bytes(),
        &validation,
    )?;

    Ok(decoded.claims)
}

/// 检查权限
pub async fn check_permission(
    &self,
    user_id: &str,

```



```

        case_id: &str,
        required_permission: &str,
    ) -> Result<bool, AuthError> {
        // 1. 获取用户角色
        let roles = self.get_user_roles(user_id).await?;

        // 2. 检查是否是管理员
        if roles.contains(&"admin".to_string()) {
            return Ok(true);
        }

        // 3. 检查案件权限
        let case_role = self.get_case_role(user_id, case_id).await?;

        match case_role.as_deref() {
            Some("owner") => Ok(true), // 案件所有者
            Some("member") => {
                // 检查角色权限
                self.has_permission_in_roles(&roles, required_permission).await
            },
            Some("viewer") => Ok(false), // 访客无编辑权限
            None => Ok(false),
        }
    }
}

/// 登录响应
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct LoginResponse {
    pub user: UserInfo,
    pub access_token: String,
    pub refresh_token: String,
    pub expires_in: i64,
}

/// Token 响应
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TokenResponse {
    pub access_token: String,
    pub expires_in: i64,
}

/// 用户信息
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct UserInfo {
    pub id: String,
    pub username: String,

```

```
pub email: String,  
pub real_name: Option<String>,  
pub avatar_url: Option<String>,  
pub roles: Vec<String>,  
}
```

四、API 接口设计

4.1 认证接口

方法	路径	说明
POST	/api/v1/auth/login	用户登录
POST	/api/v1/auth/logout	用户登出
POST	/api/v1/auth/refresh	刷新 Token
POST	/api/v1/auth/register	用户注册
POST	/api/v1/auth/forgot-password	忘记密码
POST	/api/v1/auth/reset-password	重置密码
GET	/api/v1/auth/me	获取当前用户信息
PUT	/api/v1/auth/password	修改密码

4.2 登录接口示例

请求：

```
POST /api/v1/auth/login  
Content-Type: application/json  
  
{
```

```
"username": "zhangsan",
"password": "password123",
"remember_me": true
}
```

响应:

```
{
  "code": 200,
  "message": "success",
  "data": {
    "user": {
      "id": "user_001",
      "username": "zhangsan",
      "email": "zhangsan@example.com",
      "real_name": "张三",
      "avatar_url": null,
      "roles": ["host_lawyer"]
    },
    "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "expires_in": 3600
  }
}
```

4.3 权限检查中间件

```
// src/middleware/auth.rs
use axum::{
    extract::State,
    http::{Request, StatusCode},
    middleware::Next,
    response::Response,
};

/// 认证中间件
pub async fn auth_middleware<B>(
    mut request: Request<B>,
    next: Next<B>,
) -> Result<Response, StatusCode> {
    // 从 Header 获取 Token
    let auth_header = request
        .headers()
```

```

        .get(http::header::AUTHORIZATION)
        .and_then(|h| h.to_str().ok())
        .and_then(|s| s.strip_prefix("Bearer "));

let token = auth_header.ok_or(StatusCode::UNAUTHORIZED)?;

// 验证 Token
let state = request.extensions().get::<Arc<AppState>>().unwrap();
let claims = state.auth_service.verify_token(token)
    .map_err(|_| StatusCode::UNAUTHORIZED)?;

// 将用户信息注入请求
request.extensions_mut().insert(claims);

Ok(next.run(request).await)
}

/// 权限检查中间件
pub async fn permission_middleware<B>(
    required_permission: str,
    mut request: Request<B>,
    next: Next<B>,
) -> Result<Response, StatusCode> {
    let claims = request.extensions().get::<Claims>()
        .ok_or(StatusCode::UNAUTHORIZED)?;

    let case_id = request.extensions().get::<String>()
        .ok_or(StatusCode::BAD_REQUEST)?;

    let state = request.extensions().get::<Arc<AppState>>().unwrap();

    let has_permission = state.auth_service
        .check_permission(&claims.sub, case_id, required_permission)
        .await
        .map_err(|_| StatusCode::INTERNAL_SERVER_ERROR)?;

    if !has_permission {
        return Err(StatusCode::FORBIDDEN);
    }

    Ok(next.run(request).await)
}

```

五、前端实现

5.1 登录组件

```
// frontend/src/pages/Login.rs

#[component]
pub fn LoginPage() -> Element {
    let mut username = use_signal(|| String::new());
    let mut password = use_signal(|| String::new());
    let mut remember_me = use_signal(|| false);
    let mut error = use_signal(|| None::<String>);
    let mut loading = use_signal(|| false);

    let mut navigate = use_navigator();

    let handle_login = move |_| {
        let username = username();
        let password = password();
        let remember = remember_me();

        loading.set(true);
        error.set(None);

        spawn(async move {
            match login_api(&username, &password, remember).await {
                Ok(response) => {
                    // 保存 Token
                    set_access_token(&response.access_token);
                    set_refresh_token(&response.refresh_token);
                    set_user_info(&response.user);

                    // 跳转到首页
                    navigate.push("/");
                }
                Err(e) => {
                    error.set(Some(e.to_string()));
                }
            }
        });
        loading.set(false);
    };

    rsx! {
```

```

div { class: "login-page",
  div { class: "login-container",
    h1 { "法律认知引擎" }

    if let Some(err) = error() {
      div { class: "error-message", {err} }
    }

    input {
      r#type: "text",
      placeholder: "用户名/邮箱",
      value: username(),
      oninput: move |e| username.set(e.value()),
    }

    input {
      r#type: "password",
      placeholder: "密码",
      value: password(),
      oninput: move |e| password.set(e.value()),
    }

    label { class: "checkbox-label",
      input {
        r#type: "checkbox",
        checked: remember_me(),
        onchange: move |e| remember_me.set(e.checked()),
      }
      "记住登录"
    }

    button {
      class: "login-btn primary",
      disabled: loading(),
      onclick: handle_login,
      if loading() { "登录中..." } else { "登录" }
    }

    div { class: "login-links",
      Link { to: "/forgot-password", "忘记密码" }
      Link { to: "/register", "注册账号" }
    }
  }
}

```

5.2 权限保护路由

```
// frontend/src/router.rs

#[component]
pub fn ProtectedRoute(children: Element, required_permission: Option<String>) -> Element {
    let user = use_context::<Signal<Option<UserInfo>>>>();
    let mut navigate = use_navigator();

    // 检查是否登录
    if user().is_none() {
        use_effect(move || {
            navigate.push("/login");
        });
        return rsx! { div { "跳转中..." } };
    }

    // 检查权限 (如果有要求)
    if let Some(permission) = required_permission {
        if !user().unwrap().permissions.contains(&permission) {
            return rsx! {
                div { class: "no-permission",
                    h1 { "无权限访问" }
                    p { "您没有访问此页面的权限" }
                }
            };
        }
    }

    children
}

// 使用示例
#[component]
pub fn App() -> Element {
    rsx! {
        Router::<Route> {}
    }
}

#[derive(Clone, Routable, PartialEq)]
pub enum Route {
    #[layout(ProtectedLayout)]
    #[route("/")]
    Home {},
    #[route("/cases/:id")]
}
```

```
        CaseDetail { id: String },
    }
    #[end_layout]

    #[route("/login")]
    Login {},
    #[route("/register")]
    Register {},
}
```

六、安全设计

6.1 密码安全

```
// 密码加密 (Argon2)
use argon2::{PasswordHasher, Algorithm, Params, Version};

pub fn hash_password(password: &str) -> Result<String, anyhow::Error> {
    let salt = SaltString::generate(&mut OsRng);

    let argon2 = Argon2::new(
        Algorithm::Argon2id,
        Version::V0x13,
        Params::new(19456, 2, 1, None)?, // 内存 19MB, 2 次迭代, 1 并行度
    );

    let hash = argon2.hash_password(password.as_bytes(), &salt)?;
    Ok(hash.to_string())
}
```

6.2 Token 安全

措施	说明
HTTPS	所有通信加密
短期 Access Token	1 小时有效期

措施	说明
长期 Refresh Token	7 天有效期，可撤销
Token 黑名单	登出时加入黑名单
IP 绑定（可选）	Token 与 IP 绑定

6.3 防爆破

```
// 登录失败限制
pub async fn check_login_attempts(
    username: &str,
    ip_address: &str,
) -> Result<(), AuthError> {
    // 检查用户名级别失败次数
    let user_failures = get_failed_attempts_by_user(username).await?;
    if user_failures >= 5 {
        return Err(AuthError::TooManyAttempts);
    }

    // 检查 IP 级别失败次数
    let ip_failures = get_failed_attempts_by_ip(ip_address).await?;
    if ip_failures >= 10 {
        return Err(AuthError::TooManyAttempts);
    }

    Ok(())
}
```

七、总结

7.1 核心设计

模块	技术选型	说明
密码加密	Argon2	最新密码哈希算法
Token	JWT (HS256)	无状态认证
权限模型	RBAC	基于角色的访问控制
数据权限	案件成员表	案件级访问控制

7.2 API 清单

接口	方法	说明
/auth/login	POST	登录
/auth/logout	POST	登出
/auth/refresh	POST	刷新 Token
/auth/register	POST	注册
/auth/me	GET	当前用户
/auth/password	PUT	修改密码

文档版本：v1.0

最后更新：2026 年 3 月 14 日

开发负责人：王舟

邮箱 : main@mailswedevevs.org

电话: 15378391447

开发负责人: 王舟 | 电话: 15378391447 | 邮箱: main@mailswedevevs.org